DA3408 AI Operations Assignment 3

Prabhav Gupta DA24B018

Part 1 — Setup and Dataset

I implemented the same data cleaning pipeline in Apache Spark (PySpark 3.5.3) and Ray Data (Ray 2.59).

The dataset contains NYC yellow taxi trips from January to April 2023. It has 12,672,737 rows, is 197 MB as Parquet and uses about 1.84 GB in memory. The zone lookup file has 265 rows. All experiments were run on one VirtualBox VM with 4 cores and 5.8 GB RAM.

Both frameworks used two workers with 2 cores and 1 GB RAM each. Only one framework was run at a time.

Code, results and screenshots: `https://github.com/shuriken17/DA3408-Assignment-3`

Part 2 — Pipeline and Output Match

The pipeline reads the data, removes invalid rows, joins the zone table, calculates average speed, performs a group-by join and writes the result to Parquet.

Cleaning reduced the data from 12,672,737 to 11,698,059 rows, removing 7.7%. No exact duplicates were found.

Average speed is set to null for trips shorter than 60 seconds or above 100 mph and is rounded to two decimals. Join A is a broadcast join, while Join B groups trips by pickup zone, date and hour.

The Spark and Ray outputs were compared using schema, row count, column checksums and the checksum of the complete sorted row set. All outputs had 11,698,059 rows and matched. Timestamps were converted to microseconds before comparison because Spark and Ray used different timestamp precision.

Part 3 — Performance Benchmark

Each framework was tested using Python UDF, pandas UDF and native built-in functions. Each mode was run three times.

Framework	Mode	Min (s)	Median (s)	Max (s)	CPU peak (%)	CPU mean (%)	Mem. peak (MB)
Spark	Python	110.9	126.5	128.2	99	55	5128
Spark	Pandas	98.8	108.8	214.1	96	55	5340
Spark	Native	93.6	98.6	147.4	100	55	4938
Ray	Python	341.6	381.5	396.4	99	57	5664
Ray	Pandas	205.8	246.9	965.3	100	52	5705
Ray	Native	151.8	381.3	390.2	97	49	5660

Table 1: End-to-end benchmark results. Mem. peak is whole-VM memory used.

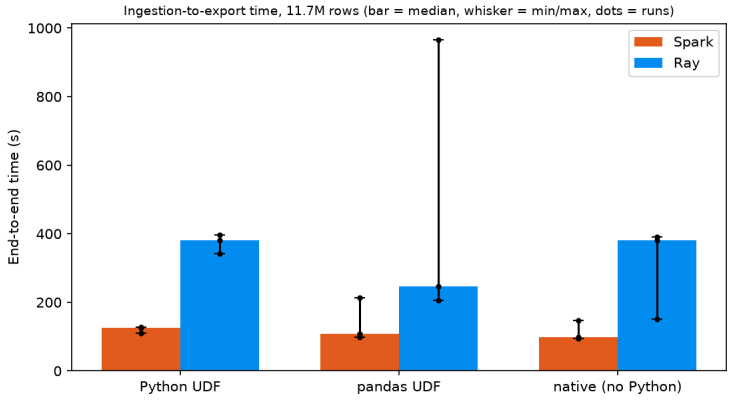


Figure 1: End-to-end execution time.

Spark was faster in every mode. Its median time was 2.3–3.9 times lower than Ray. Spark's runs also varied less in absolute time. Ray's pandas mode had one run of 965 seconds. System memory peaked at 4.9–5.3 GB during Spark runs and about 5.7 GB during Ray runs, on a VM with 5.8 GB.

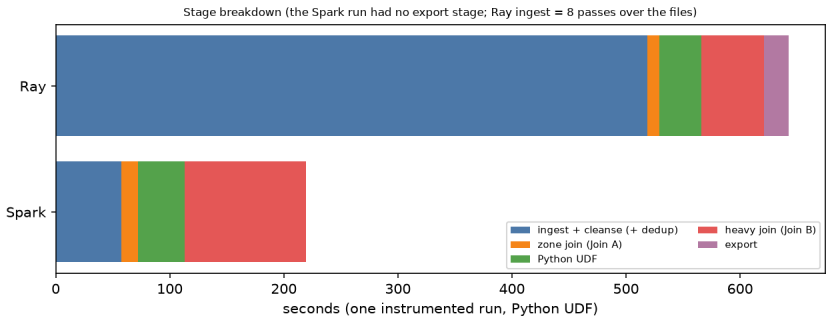


Figure 2: Stage breakdown for one Python-UDF run.

Ray spent 519 of 642 seconds on ingestion, cleaning and duplicate removal, compared with 58 seconds for Spark. Ray's group-by shuffle did not finish on one month of data in 5 minutes

and later exceeded the VM's memory, so duplicate removal and Join B used 8 hash buckets. This required reading the input files again.

Ray's Join B took 55 seconds compared with 106 seconds for Spark, but the implementations were different because Ray avoided the shuffle.

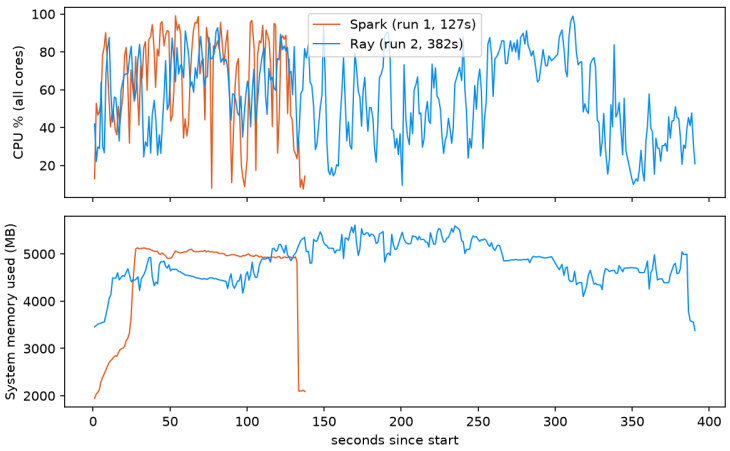


Figure 3: CPU and memory over time.

Part 4 — Python UDF Overhead

The cleaned data was cached and only the speed calculation was timed. All six cases produced the same sum: 13,739,637,981. The Ray test adds up the speed column on the driver while Spark adds it up in the cluster, so the work is not exactly equal, and each number comes from three runs.

Framework	Python (s)	Pandas (s)	Native (s)	Python OH (s)	Pandas OH (s)
Spark	10.2	5.4	1.3	+8.9	+4.2
Ray	2.5	2.2	1.2	+1.3	+1.0

Table 2: UDF-only time and overhead.

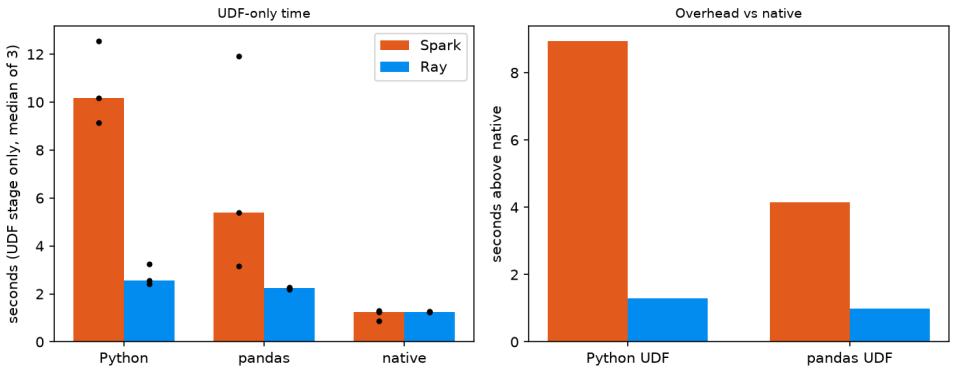


Figure 4: UDF-only time and overhead.

Spark's Python UDF adds JVM-to-Python communication, giving 8.9 seconds of overhead. The pandas UDF reduces this to 4.2 seconds. Ray's Python and pandas overheads were only 1.3 and 1.0 seconds.

Thus, Spark's Python UDF overhead was about seven times higher than Ray's. In the end-to-end benchmark, the Spark Python UDF added 27.9 seconds compared with native functions.

Part 5 — Winner and Recommendation

Winner: Spark.

Spark was faster, with runs that varied less in absolute time, for this workload. Its median times were 98.6–126.5 seconds, compared with 246.9–381.5 seconds for Ray.

BI-first project: I would choose **Spark**. The workload mainly uses filters, joins and aggregations, which Spark handled well. Its native mode was also the fastest at 98.6 seconds.

AI-first project: I would choose **Ray**. Its Python UDF overhead was much lower than Spark's (1.3 seconds vs. 8.9 seconds). This makes Ray more suitable when a pipeline contains a large amount of custom Python processing.

Therefore, the overall winner for this benchmark is **Spark**, while Ray has an advantage for workloads that rely heavily on custom Python functions.

Part 6 — Limitations and Performance Tuning Note

The experiments used one VM, three runs per mode, and no page-cache clearing. The stage breakdown came from one run. Memory pressure also caused some runs to be killed by `systemd-oomd`, which was stopped during the benchmark.

Performance Tuning Note: I used Claude to help translate the pipeline between Spark and Ray and suggest optimizations. For Ray, it suggested replacing the full group-by used for duplicate removal with 8 hash buckets because the group-by ran out of memory. It also suggested reducing the UDF stage from 4 blocks to 2 because of memory pressure. It also suggested turning off Spark adaptive query execution so that Spark's 8 shuffle partitions

match Ray's 8 buckets, and writing Spark timestamps in microseconds to match Ray. I did not measure the effect of either change separately. These changes allowed the pipeline to finish on the available VM.

All benchmark numbers in this report are from my own runs.